

E-commerce

August 22, 2026

1 E-commerce sales analysis

This notebook combines two years of Online Retail transactions, checks and cleans the records, measures sales performance, builds RFM customer segments, and exports tables for reporting. Each row in the source data represents an item line within an invoice; a single invoice can contain multiple rows.

```
[4]: import pandas as pd
```

1.1 Load the two annual source sheets

The Online Retail workbook contains transaction-level data: each row is one product line in an invoice, with the invoice number, product code and description, quantity, date, unit price, customer ID, and country. The next cells read the 2009–2010 sheet and then the 2010–2011 sheet into separate Pandas DataFrames before they are combined.

```
[5]: file_path = "online_retail_II.xlsx"

df_2009_2010 = pd.read_excel(
    file_path,
    sheet_name="Year 2009-2010"
)
```

```
[6]: df_2009_2010.head()
```

```
[6]: Invoice StockCode      Description  Quantity \
0  489434      85048  15CM CHRISTMAS GLASS BALL 20 LIGHTS      12
1  489434      79323P                PINK CHERRY LIGHTS      12
2  489434      79323W                WHITE CHERRY LIGHTS      12
3  489434      22041          RECORD FRAME 7" SINGLE SIZE      48
4  489434      21232        STRAWBERRY CERAMIC TRINKET BOX      24
```

```
InvoiceDate  Price  Customer ID      Country
0 2009-12-01 07:45:00  6.95      13085.0  United Kingdom
1 2009-12-01 07:45:00  6.75      13085.0  United Kingdom
2 2009-12-01 07:45:00  6.75      13085.0  United Kingdom
3 2009-12-01 07:45:00  2.10      13085.0  United Kingdom
4 2009-12-01 07:45:00  1.25      13085.0  United Kingdom
```

The preceding cell previews the 2009–2010 sheet. The next two cells load and preview the 2010–2011 sheet, confirming that both annual imports have the expected structure before they are combined.

```
[7]: df_2010_2011 = pd.read_excel(
      file_path,
      sheet_name="Year 2010-2011"
    )
```

```
[8]: df_2010_2011.head()
```

```
[8]: Invoice StockCode      Description  Quantity \
0  536365      85123A  WHITE HANGING HEART T-LIGHT HOLDER      6
1  536365      71053                WHITE METAL LANTERN      6
2  536365      84406B      CREAM CUPID HEARTS COAT HANGER      8
3  536365      84029G  KNITTED UNION FLAG HOT WATER BOTTLE      6
4  536365      84029E      RED WOOLLY HOTTIE WHITE HEART.      6

      InvoiceDate  Price  Customer ID      Country
0  2010-12-01 08:26:00    2.55      17850.0  United Kingdom
1  2010-12-01 08:26:00    3.39      17850.0  United Kingdom
2  2010-12-01 08:26:00    2.75      17850.0  United Kingdom
3  2010-12-01 08:26:00    3.39      17850.0  United Kingdom
4  2010-12-01 08:26:00    3.39      17850.0  United Kingdom
```

1.2 Combine and inspect the source data

The two annual DataFrames are appended row by row. `ignore_index=True` creates one continuous row index. The following checks display the table size, its first and last records, verify that no rows were lost during the append, and inspect column names, types, summary statistics, and example records.

```
[9]: df = pd.concat(
      [df_2009_2010, df_2010_2011],
      ignore_index=True
    )
```

```
[10]: df.shape
```

```
[10]: (1067371, 8)
```

```
[11]: df.head()
```

```
[11]: Invoice StockCode      Description  Quantity \
0  489434      85048  15CM CHRISTMAS GLASS BALL 20 LIGHTS      12
1  489434      79323P                PINK CHERRY LIGHTS      12
2  489434      79323W                WHITE CHERRY LIGHTS      12
3  489434      22041      RECORD FRAME 7" SINGLE SIZE      48
4  489434      21232      STRAWBERRY CERAMIC TRINKET BOX      24
```

	InvoiceDate	Price	Customer ID	Country
0	2009-12-01 07:45:00	6.95	13085.0	United Kingdom
1	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
2	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
3	2009-12-01 07:45:00	2.10	13085.0	United Kingdom
4	2009-12-01 07:45:00	1.25	13085.0	United Kingdom

```
[12]: df.tail()
```

```
[12]:
```

	Invoice	StockCode	Description	Quantity	\
1067366	581587	22899	CHILDREN'S APRON DOLLY GIRL	6	
1067367	581587	23254	CHILDRENS CUTLERY DOLLY GIRL	4	
1067368	581587	23255	CHILDRENS CUTLERY CIRCUS PARADE	4	
1067369	581587	22138	BAKING SET 9 PIECE RETROSPOT	3	
1067370	581587	POST	POSTAGE	1	

	InvoiceDate	Price	Customer ID	Country
1067366	2011-12-09 12:50:00	2.10	12680.0	France
1067367	2011-12-09 12:50:00	4.15	12680.0	France
1067368	2011-12-09 12:50:00	4.15	12680.0	France
1067369	2011-12-09 12:50:00	4.95	12680.0	France
1067370	2011-12-09 12:50:00	18.00	12680.0	France

```
[13]: len(df_2009_2010) + len(df_2010_2011) == len(df)
```

```
[13]: True
```

```
[14]: df.columns
```

```
[14]: Index(['Invoice', 'StockCode', 'Description', 'Quantity', 'InvoiceDate',
          'Price', 'Customer ID', 'Country'],
          dtype='object')
```

```
[15]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1067371 entries, 0 to 1067370
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Invoice          1067371 non-null  object
1   StockCode       1067371 non-null  object
2   Description     1062989 non-null  object
3   Quantity        1067371 non-null  int64
4   InvoiceDate     1067371 non-null  datetime64[ns]
5   Price           1067371 non-null  float64
6   Customer ID    824364 non-null   float64
```

```

7    Country      1067371 non-null object
dtypes: datetime64[ns](1), float64(2), int64(1), object(4)
memory usage: 65.1+ MB

```

```
[16]: df.describe()
```

```

[16]:      Quantity      InvoiceDate      Price \
count  1.067371e+06      1067371  1.067371e+06
mean    9.938898e+00  2011-01-02 21:13:55.394028544  4.649388e+00
min    -8.099500e+04      2009-12-01 07:45:00 -5.359436e+04
25%     1.000000e+00      2010-07-09 09:46:00  1.250000e+00
50%     3.000000e+00      2010-12-07 15:28:00  2.100000e+00
75%     1.000000e+01      2011-07-22 10:23:00  4.150000e+00
max     8.099500e+04      2011-12-09 12:50:00  3.897000e+04
std     1.727058e+02                NaN  1.235531e+02

      Customer ID
count  824364.000000
mean    15324.638504
min     12346.000000
25%     13975.000000
50%     15255.000000
75%     16797.000000
max     18287.000000
std      1697.464450

```

```
[17]: df.head()
```

```

[17]:      Invoice StockCode      Description      Quantity \
0  489434      85048  15CM CHRISTMAS GLASS BALL 20 LIGHTS      12
1  489434      79323P                PINK CHERRY LIGHTS      12
2  489434      79323W                WHITE CHERRY LIGHTS      12
3  489434      22041          RECORD FRAME 7" SINGLE SIZE      48
4  489434      21232        STRAWBERRY CERAMIC TRINKET BOX      24

      InvoiceDate      Price      Customer ID      Country
0  2009-12-01 07:45:00      6.95      13085.0  United Kingdom
1  2009-12-01 07:45:00      6.75      13085.0  United Kingdom
2  2009-12-01 07:45:00      6.75      13085.0  United Kingdom
3  2009-12-01 07:45:00      2.10      13085.0  United Kingdom
4  2009-12-01 07:45:00      1.25      13085.0  United Kingdom

```

1.3 Assess data quality

These checks count missing values and calculate the percentage of missing values in every column. They show which fields need attention before analysis.

```
[18]: df.isnull().sum()
```

```
[18]: Invoice          0
      StockCode       0
      Description    4382
      Quantity       0
      InvoiceDate     0
      Price          0
      Customer ID    243007
      Country        0
      dtype: int64
```

```
[19]: (df.isnull().mean() * 100).round(2)
```

```
[19]: Invoice          0.00
      StockCode       0.00
      Description    0.41
      Quantity       0.00
      InvoiceDate     0.00
      Price          0.00
      Customer ID    22.77
      Country        0.00
      dtype: float64
```

1.3.1 Diagnose duplicate transaction rows

This section identifies exact duplicate rows, reviews sample duplicates, and groups them by invoice and product code. The grouped count distinguishes repeated line items from likely duplicate records and helps evaluate their possible effect on revenue.

```
[20]: df.duplicated().sum()
```

```
[20]: np.int64(34335)
```

```
[21]: df[df.duplicated()].head()
```

```
[21]: Invoice StockCode Description Quantity \
371  489517    21912    VINTAGE SNAKES & LADDERS      1
383  489517    22130    PARTY CONE CHRISTMAS DECORATION      6
384  489517    22319    HAIRCLIPS FORTIES FABRIC ASSORTED     12
385  489517    21913    VINTAGE SEASIDE JIGSAW PUZZLES      1
386  489517    21821    GLITTER STAR GARLAND WITH BELLS      1

      InvoiceDate Price Customer ID Country
371  2009-12-01 11:34:00    3.75    16329.0 United Kingdom
383  2009-12-01 11:34:00    0.85    16329.0 United Kingdom
384  2009-12-01 11:34:00    0.65    16329.0 United Kingdom
385  2009-12-01 11:34:00    3.75    16329.0 United Kingdom
386  2009-12-01 11:34:00    3.75    16329.0 United Kingdom
```

```
[22]: duplicates = df[df.duplicated(keep=False)]
```

```
[23]: duplicates.sort_values(
      ["Invoice", "StockCode", "InvoiceDate"]
    ).head(20)
```

```
[23]:
```

	Invoice	StockCode	Description	Quantity	\
379	489517	21491	SET OF THREE VINTAGE GIFT WRAPS	1	
391	489517	21491	SET OF THREE VINTAGE GIFT WRAPS	1	
365	489517	21821	GLITTER STAR GARLAND WITH BELLS	1	
386	489517	21821	GLITTER STAR GARLAND WITH BELLS	1	
363	489517	21912	VINTAGE SNAKES & LADDERS	1	
371	489517	21912	VINTAGE SNAKES & LADDERS	1	
394	489517	21912	VINTAGE SNAKES & LADDERS	1	
362	489517	21913	VINTAGE SEASIDE JIGSAW PUZZLES	1	
385	489517	21913	VINTAGE SEASIDE JIGSAW PUZZLES	1	
368	489517	22130	PARTY CONE CHRISTMAS DECORATION	6	
383	489517	22130	PARTY CONE CHRISTMAS DECORATION	6	
367	489517	22319	HAIRCLIPS FORTIES FABRIC ASSORTED	12	
384	489517	22319	HAIRCLIPS FORTIES FABRIC ASSORTED	12	
388	489517	84951A	S/4 PISTACHIO LOVEBIRD COASTERS	1	
390	489517	84951A	S/4 PISTACHIO LOVEBIRD COASTERS	1	
634	489529	22028	PENNY FARTHING BIRTHDAY CARD	12	
657	489529	22028	PENNY FARTHING BIRTHDAY CARD	12	
656	489529	22036	DINOSAUR BIRTHDAY CARD	12	
658	489529	22036	DINOSAUR BIRTHDAY CARD	12	
628	489529	48129	DOOR MAT TOPIARY	1	

	InvoiceDate	Price	Customer ID	Country
379	2009-12-01 11:34:00	1.95	16329.0	United Kingdom
391	2009-12-01 11:34:00	1.95	16329.0	United Kingdom
365	2009-12-01 11:34:00	3.75	16329.0	United Kingdom
386	2009-12-01 11:34:00	3.75	16329.0	United Kingdom
363	2009-12-01 11:34:00	3.75	16329.0	United Kingdom
371	2009-12-01 11:34:00	3.75	16329.0	United Kingdom
394	2009-12-01 11:34:00	3.75	16329.0	United Kingdom
362	2009-12-01 11:34:00	3.75	16329.0	United Kingdom
385	2009-12-01 11:34:00	3.75	16329.0	United Kingdom
368	2009-12-01 11:34:00	0.85	16329.0	United Kingdom
383	2009-12-01 11:34:00	0.85	16329.0	United Kingdom
367	2009-12-01 11:34:00	0.65	16329.0	United Kingdom
384	2009-12-01 11:34:00	0.65	16329.0	United Kingdom
388	2009-12-01 11:34:00	2.55	16329.0	United Kingdom
390	2009-12-01 11:34:00	2.55	16329.0	United Kingdom
634	2009-12-01 11:51:00	0.42	17984.0	United Kingdom
657	2009-12-01 11:51:00	0.42	17984.0	United Kingdom
656	2009-12-01 11:51:00	0.42	17984.0	United Kingdom

```
658 2009-12-01 11:51:00 0.42      17984.0 United Kingdom
628 2009-12-01 11:51:00 6.75      17984.0 United Kingdom
```

```
[24]: duplicates.groupby(
        ["Invoice", "StockCode"]
    ).size().sort_values(ascending=False).head(20)
```

```
[24]: Invoice  StockCode
555524    22698         20
         22697         12
536412    21448         10
537646    22585         10
537224    70007         10
536464    22866          8
537154    22727          8
537202    22147          8
502660    17021          8
572861    22775          8
536874    22866          6
537051    22727          6
538514    21756          6
536796    21967          6
         22376          6
525065    20894          6
537421    22585          6
537422    22585          6
537144    22553          6
533841    22906          6
dtype: int64
```

1.3.2 Measure the effect of duplicates

Revenue is calculated per transaction line as quantity $\times$ unit price. The code then compares total revenue before and after removing exact duplicate rows, reporting both the currency difference and its percentage of the original total. A deduplicated working copy, `df_clean`, is created for the remaining analysis.

```
[25]: df["Revenue"] = df["Quantity"] * df["Price"]
```

```
[26]: total_revenue_with_duplicates = df["Revenue"].sum()
total_revenue_with_duplicates
```

```
[26]: np.float64(19287250.568)
```

```
[27]: df_no_duplicates = df.drop_duplicates()

total_revenue_without_duplicates = df_no_duplicates["Revenue"].sum()
```

```
total_revenue_without_duplicates
```

```
[27]: np.float64(18855533.698)
```

```
[28]: difference = (  
        total_revenue_with_duplicates  
        - total_revenue_without_duplicates  
    )  
  
difference
```

```
[28]: np.float64(431716.87000000104)
```

```
[29]: percentage_difference = (  
        difference / total_revenue_with_duplicates  
    ) * 100  
  
percentage_difference
```

```
[29]: np.float64(2.2383536133256556)
```

```
[30]: df_clean = df.drop_duplicates().copy()
```

```
[31]: df_clean.shape
```

```
[31]: (1033036, 9)
```

```
[32]: df_clean.duplicated().sum()
```

```
[32]: np.int64(0)
```

1.4 Clean and profile key fields

The following cells examine transactions without a customer identifier and show where they occur. These rows remain in `df_clean` for sales analysis, but cannot be assigned to an individual customer in the later RFM analysis.

```
[33]: df_clean[df_clean["Customer ID"].isna()].head(10)
```

```
[33]:
```

	Invoice	StockCode	Description	Quantity	\
263	489464	21733	85123a mixed	-96	
283	489463	71477	short	-240	
284	489467	85123A	21733 mixed	-192	
470	489521	21646	NaN	-50	
577	489525	85226C	BLUE PULL BACK RACING CAR	1	
578	489525	85227	SET/6 3D KIT CARDS FOR KIDS	1	
1055	489548	22271	FELTCRAFT DOLL ROSIE	1	
1056	489548	22254	FELT TOADSTOOL LARGE	12	

1057	489548	22273	FELTCRAFT DOLL MOLLY	3
1058	489548	22195	LARGE HEART MEASURING SPOONS	1

	InvoiceDate	Price	Customer ID	Country	Revenue
263	2009-12-01 10:52:00	0.00	NaN	United Kingdom	-0.00
283	2009-12-01 10:52:00	0.00	NaN	United Kingdom	-0.00
284	2009-12-01 10:53:00	0.00	NaN	United Kingdom	-0.00
470	2009-12-01 11:44:00	0.00	NaN	United Kingdom	-0.00
577	2009-12-01 11:49:00	0.55	NaN	United Kingdom	0.55
578	2009-12-01 11:49:00	0.85	NaN	United Kingdom	0.85
1055	2009-12-01 12:32:00	2.95	NaN	United Kingdom	2.95
1056	2009-12-01 12:32:00	1.25	NaN	United Kingdom	15.00
1057	2009-12-01 12:32:00	2.95	NaN	United Kingdom	8.85
1058	2009-12-01 12:32:00	1.65	NaN	United Kingdom	1.65

```
[34]: df_clean[df_clean["Customer ID"].isna()]["Country"].value_counts().head(10)
```

```
[34]: Country
United Kingdom      232206
EIRE                 1653
Hong Kong           360
Unspecified         231
France              128
Switzerland         117
United Arab Emirates 114
Portugal            114
Bahrain              67
Israel               47
Name: count, dtype: int64
```

1.4.1 Remove invalid or free-price rows

The code counts and inspects zero or negative prices, checks which product descriptions and invoice types are involved, then keeps only rows with a positive price. This prevents zero- or negative-priced records from distorting revenue measures. Negative quantities are intentionally retained because they can represent returns or cancellations.

```
[35]: (df_clean["Price"] <= 0).sum()
```

```
[35]: np.int64(6019)
```

```
[36]: df_clean[df_clean["Price"] <= 0].head(20)
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	\
263	489464	21733	85123a mixed	-96	2009-12-01 10:52:00	
283	489463	71477	short	-240	2009-12-01 10:52:00	
284	489467	85123A	21733 mixed	-192	2009-12-01 10:53:00	
470	489521	21646	NaN	-50	2009-12-01 11:44:00	

3114	489655	20683		NaN	-44	2009-12-01	17:26:00
3161	489659	21350		NaN	230	2009-12-01	17:39:00
3162	489660	35956		lost	-1043	2009-12-01	17:43:00
3168	489663	35605A		damages	-117	2009-12-01	18:02:00
3731	489781	84292		NaN	17	2009-12-02	11:45:00
4296	489806	18010		NaN	-770	2009-12-02	12:42:00
4538	489820	21133		invcd as 84879?	-720	2009-12-02	13:23:00
4566	489821	85049G		NaN	-240	2009-12-02	13:25:00
4674	489825	22076	6 RIBBONS EMPIRE		12	2009-12-02	13:34:00
5904	489861	DOT	DOTCOM POSTAGE		1	2009-12-02	14:50:00
6378	489882	35751C		NaN	12	2009-12-02	16:22:00
6555	489898	79323G		NaN	954	2009-12-03	09:40:00
6556	489899	79323GR		sold as gold	-954	2009-12-03	09:41:00
6576	489901	21098		NaN	-200	2009-12-03	09:47:00
6581	489903	21166		NaN	48	2009-12-03	09:57:00
6781	489998	48185	DOOR MAT FAIRY CAKE		2	2009-12-03	11:19:00

	Price	Customer ID	Country	Revenue
263	0.0	NaN	United Kingdom	-0.0
283	0.0	NaN	United Kingdom	-0.0
284	0.0	NaN	United Kingdom	-0.0
470	0.0	NaN	United Kingdom	-0.0
3114	0.0	NaN	United Kingdom	-0.0
3161	0.0	NaN	United Kingdom	0.0
3162	0.0	NaN	United Kingdom	-0.0
3168	0.0	NaN	United Kingdom	-0.0
3731	0.0	NaN	United Kingdom	0.0
4296	0.0	NaN	United Kingdom	-0.0
4538	0.0	NaN	United Kingdom	-0.0
4566	0.0	NaN	United Kingdom	-0.0
4674	0.0	16126.0	United Kingdom	0.0
5904	0.0	NaN	United Kingdom	0.0
6378	0.0	NaN	United Kingdom	0.0
6555	0.0	NaN	United Kingdom	0.0
6556	0.0	NaN	United Kingdom	-0.0
6576	0.0	NaN	United Kingdom	-0.0
6581	0.0	NaN	United Kingdom	0.0
6781	0.0	15658.0	United Kingdom	0.0

```
[37]: (df_clean["Price"] == 0).sum()
```

```
[37]: np.int64(6014)
```

```
[38]: df_clean.loc[
    df_clean["Price"] == 0,
    "Description"
].value_counts(dropna=False).head(30)
```

```
[38]: Description
      NaN                                4275
      check                             160
      ?                                 90
      damages                           83
      damaged                           81
      found                             28
      missing                           27
      sold as set on dotcom              20
      Damaged                           17
      adjustment                         16
      OWL DOORSTOP                       14
      dotcom                             12
      POLYESTER FILLER PAD 45x45cm       12
      POLYESTER FILLER PAD 40x40cm       10
      Found                              9
      FRENCH BLUE METAL DOOR SIGN 1      9
      smashed                            9
      Unsaleable, destroyed.              9
      thrown away                         9
      checked                             8
      RED KITCHEN SCALES                  8
      RECIPE BOX PANTRY YELLOW DESIGN    8
      IVORY KITCHEN SCALES                8
      amazon                             8
      PICNIC BASKET WICKER LARGE          8
      FRENCH BLUE METAL DOOR SIGN 8      8
      BOX OF 24 COCKTAIL PARASOLS        8
      MINT KITCHEN SCALES                 7
      ENAMEL WASH BOWL CREAM              7
      FRENCH BLUE METAL DOOR SIGN No     7
      Name: count, dtype: int64
```

```
[39]: df_clean.loc[
      df_clean["Price"] == 0,
      "Invoice"
    ].astype(str).str.startswith("C").value_counts()
```

```
[39]: Invoice
      False      6014
      Name: count, dtype: int64
```

```
[40]: df_clean = df_clean[df_clean["Price"] > 0].copy()
```

```
[41]: (df_clean["Price"] <= 0).sum()
```

```
[41]: np.int64(0)
```

```
[42]: df_clean.shape
```

```
[42]: (1027017, 9)
```

1.4.2 Identify cancellation transactions

Negative quantities are inspected and compared with invoice numbers beginning with C. A Boolean `IsCancellation` flag is then created from that invoice-number convention. The counts, cross-tabulation, and final filter check whether cancellation status aligns with negative quantities and whether any unusual cases exist.

```
[43]: (df_clean["Quantity"] < 0).sum()
```

```
[43]: np.int64(19103)
```

```
[44]: df_clean[df_clean["Quantity"] < 0].head(20)
```

```
[44]:
```

	Invoice	StockCode	Description	Quantity	\
178	C489449	22087	PAPER BUNTING WHITE LACE	-12	
179	C489449	85206A	CREAM FELT EASTER EGG BASKET	-6	
180	C489449	21895	POTTING SHED SOW 'N' GROW SET	-4	
181	C489449	21896	POTTING SHED TWINE	-6	
182	C489449	22083	PAPER CHAIN KIT RETRO SPOT	-12	
183	C489449	21871	SAVE THE PLANET MUG	-12	
184	C489449	84946	ANTIQUE SILVER TEA GLASS ETCHED	-12	
185	C489449	84970S	HANGING HEART ZINC T-LIGHT HOLDER	-24	
186	C489449	22090	PAPER BUNTING RETRO SPOTS	-12	
196	C489459	90200A	PURPLE SWEETHEART BRACELET	-3	
197	C489459	90200D	PINK SWEETHEART BRACELET	-3	
198	C489459	90200B	BLACK SWEETHEART BRACELET	-3	
199	C489459	90200E	GREEN SWEETHEART BRACELET	-3	
200	C489459	90200C	BLUE SWEETHEART BRACELET	-3	
201	C489459	90185C	BLACK DIAMANTE EXPANDABLE RING	-3	
202	C489459	90185B	AMETHYST DIAMANTE EXPANDABLE RING	-3	
203	C489459	90003C	MIDNIGHT BLUE PAIR HEART HAIR SLIDE	-3	
204	C489459	90003D	CRYSTAL PAIR HEART HAIR SLIDES	-3	
205	C489459	90003B	ROSE COLOUR PAIR HEART HAIR SLIDES	-3	
206	C489459	90209C	PINK ENAMEL+GLASS HAIR COMB	-3	

	InvoiceDate	Price	Customer	ID	Country	Revenue
178	2009-12-01 10:33:00	2.95	16321.0		Australia	-35.40
179	2009-12-01 10:33:00	1.65	16321.0		Australia	-9.90
180	2009-12-01 10:33:00	4.25	16321.0		Australia	-17.00
181	2009-12-01 10:33:00	2.10	16321.0		Australia	-12.60
182	2009-12-01 10:33:00	2.95	16321.0		Australia	-35.40
183	2009-12-01 10:33:00	1.25	16321.0		Australia	-15.00
184	2009-12-01 10:33:00	1.25	16321.0		Australia	-15.00
185	2009-12-01 10:33:00	0.85	16321.0		Australia	-20.40

186	2009-12-01 10:33:00	2.95	16321.0	Australia	-35.40
196	2009-12-01 10:44:00	4.25	17592.0	United Kingdom	-12.75
197	2009-12-01 10:44:00	4.25	17592.0	United Kingdom	-12.75
198	2009-12-01 10:44:00	4.25	17592.0	United Kingdom	-12.75
199	2009-12-01 10:44:00	4.25	17592.0	United Kingdom	-12.75
200	2009-12-01 10:44:00	4.25	17592.0	United Kingdom	-12.75
201	2009-12-01 10:44:00	4.25	17592.0	United Kingdom	-12.75
202	2009-12-01 10:44:00	4.25	17592.0	United Kingdom	-12.75
203	2009-12-01 10:44:00	3.75	17592.0	United Kingdom	-11.25
204	2009-12-01 10:44:00	3.75	17592.0	United Kingdom	-11.25
205	2009-12-01 10:44:00	3.75	17592.0	United Kingdom	-11.25
206	2009-12-01 10:44:00	2.10	17592.0	United Kingdom	-6.30

```
[45]: df_clean.loc[
        df_clean["Quantity"] < 0,
        "Invoice"
    ].astype(str).str.startswith("C").value_counts()
```

```
[45]: Invoice
True    19103
Name: count, dtype: int64
```

```
[46]: df_clean["IsCancellation"] = (
        df_clean["Invoice"]
        .astype(str)
        .str.startswith("C")
    )
```

```
[47]: df_clean["IsCancellation"].value_counts()
```

```
[47]: IsCancellation
False    1007913
True      19104
Name: count, dtype: int64
```

```
[48]: pd.crosstab(
        df_clean["IsCancellation"],
        df_clean["Quantity"] < 0
    )
```

```
[48]: Quantity      False  True
IsCancellation
False           1007913      0
True              1  19103
```

```
[49]: df_clean[
        (df_clean["IsCancellation"]) &
        (df_clean["Quantity"] >= 0)
    ]
```

```
]

```

```
[49]: Invoice StockCode Description Quantity InvoiceDate Price \
76799 C496350 M Manual 1 2010-02-01 08:24:00 373.57

Customer ID Country Revenue IsCancellation
76799 NaN United Kingdom 373.57 True
```

1.4.3 Check missing product descriptions

These diagnostics list transactions with a missing product description, identify their product codes, and count them. No imputation is applied: the records are preserved for overall sales calculations, while product summaries that group by description will naturally exclude missing descriptions.

```
[50]: df_clean[df_clean["Description"].isna()].head(20)
```

```
[50]: Empty DataFrame
Columns: [Invoice, StockCode, Description, Quantity, InvoiceDate, Price,
Customer ID, Country, Revenue, IsCancellation]
Index: []
```

```
[51]: df_clean[df_clean["Description"].isna()]["StockCode"].value_counts().head(20)
```

```
[51]: Series([], Name: count, dtype: int64)
```

```
[52]: df_clean["Description"].isna().sum()
```

```
[52]: np.int64(0)
```

1.4.4 Summarize customer and product coverage

The code counts unique customers and products, quantifies transactions without a customer ID, and totals the revenue from identified versus anonymous customers. It then creates a customer-only subset for customer-level analysis. This initial subset is later rebuilt after the date features are added.

```
[53]: df_clean["Customer ID"].nunique()
```

```
[53]: 5939
```

```
[54]: df_clean["StockCode"].nunique()
```

```
[54]: 4932
```

```
[55]: df_clean["Customer ID"].isna().sum()
```

```
[55]: np.int64(229202)
```

```
[56]: df_clean["Customer ID"].isna().mean() * 100
```

```
[56]: np.float64(22.31725472898696)
```

```
[57]: df_clean.groupby(  
        df_clean["Customer ID"].isna()  
    )["Revenue"].sum()
```

```
[57]: Customer ID  
False    1.628999e+07  
True     2.724219e+06  
Name: Revenue, dtype: float64
```

1.4.5 Create an initial customer-only subset

This cell keeps only rows with a customer ID. It creates an initial customer-linked subset, which is rebuilt later after the date features have been added for RFM analysis.

```
[58]: df_customers = df_clean[df_clean["Customer ID"].notna()].copy()
```

1.5 Create time features

InvoiceDate is converted to a true datetime value. Year, numeric month, month name, day of month, weekday name, and hour are extracted so sales can be grouped by different time periods. The final preview confirms the new columns.

```
[59]: df_clean["InvoiceDate"] = pd.to_datetime(df_clean["InvoiceDate"])
```

```
[60]: df_clean["Year"] = df_clean["InvoiceDate"].dt.year
```

```
[61]: df_clean["Month"] = df_clean["InvoiceDate"].dt.month
```

```
[62]: df_clean["Month_Name"] = df_clean["InvoiceDate"].dt.month_name()
```

```
[63]: df_clean["Day"] = df_clean["InvoiceDate"].dt.day
```

```
[64]: df_clean["Day_Name"] = df_clean["InvoiceDate"].dt.day_name()
```

```
[65]: df_clean["Hour"] = df_clean["InvoiceDate"].dt.hour
```

```
[66]: df_clean.head()
```

```
[66]: Invoice StockCode      Description  Quantity  \  
0  489434      85048  15CM CHRISTMAS GLASS BALL 20 LIGHTS      12  
1  489434      79323P          PINK CHERRY LIGHTS      12  
2  489434      79323W          WHITE CHERRY LIGHTS      12  
3  489434      22041      RECORD FRAME 7" SINGLE SIZE      48  
4  489434      21232  STRAWBERRY CERAMIC TRINKET BOX      24
```

```
InvoiceDate  Price  Customer ID      Country  Revenue  \  
0 2009-12-01 07:45:00    6.95      13085.0  United Kingdom    83.4
```

1	2009-12-01 07:45:00	6.75	13085.0	United Kingdom	81.0
2	2009-12-01 07:45:00	6.75	13085.0	United Kingdom	81.0
3	2009-12-01 07:45:00	2.10	13085.0	United Kingdom	100.8
4	2009-12-01 07:45:00	1.25	13085.0	United Kingdom	30.0

	IsCancellation	Year	Month	Month_Name	Day	Day_Name	Hour
0	False	2009	12	December	1	Tuesday	7
1	False	2009	12	December	1	Tuesday	7
2	False	2009	12	December	1	Tuesday	7
3	False	2009	12	December	1	Tuesday	7
4	False	2009	12	December	1	Tuesday	7

1.5.1 Confirm the analysis period

The earliest and latest transaction timestamps define the period covered by the cleaned dataset. These dates also provide the reference point for customer recency later in the notebook.

```
[67]: df_clean["InvoiceDate"].min()
```

```
[67]: Timestamp('2009-12-01 07:45:00')
```

```
[68]: df_clean["InvoiceDate"].max()
```

```
[68]: Timestamp('2011-12-09 12:50:00')
```

1.6 Calculate headline sales metrics

Net revenue includes all retained transaction lines, so cancellation lines reduce it. Gross revenue excludes invoice numbers flagged as cancellations. **returned_amount** is therefore a negative adjustment rather than an absolute return value. Order count is the number of distinct invoice identifiers (including cancellation invoices), and average order value is net revenue divided by that count. The dictionary gathers these measures in one result.

```
[69]: total_revenue = df_clean["Revenue"].sum()
```

```
total_revenue
```

```
[69]: np.float64(19014209.838)
```

```
[70]: gross_revenue = df_clean.loc[
    ~df_clean["IsCancellation"],
    "Revenue"
].sum()
```

```
gross_revenue
```

```
[70]: np.float64(20476260.448)
```

```
[71]: returned_amount = df_clean.loc[
        df_clean["IsCancellation"],
        "Revenue"
    ].sum()

    returned_amount
```

```
[71]: np.float64(-1462050.61)
```

```
[72]: total_orders = df_clean["Invoice"].nunique()

    total_orders
```

```
[72]: 48369
```

```
[73]: total_customers = df_clean["Customer ID"].nunique()

    total_customers
```

```
[73]: 5939
```

```
[74]: total_products = df_clean["StockCode"].nunique()

    total_products
```

```
[74]: 4932
```

```
[75]: average_order_value = (
        total_revenue / total_orders
    )

    average_order_value
```

```
[75]: np.float64(393.10735880419276)
```

```
[76]: metrics = {
        "Net Revenue": total_revenue,
        "Gross Revenue": gross_revenue,
        "Returned Amount": returned_amount,
        "Orders": total_orders,
        "Customers": total_customers,
        "Products": total_products,
        "Average Order Value": average_order_value
    }

    metrics
```

```
[76]: {'Net Revenue': np.float64(19014209.838),
      'Gross Revenue': np.float64(20476260.448),
      'Returned Amount': np.float64(-1462050.61),
      'Orders': 48369,
      'Customers': 5939,
      'Products': 4932,
      'Average Order Value': np.float64(393.10735880419276)}
```

1.7 Analyze sales over time

Revenue is first aggregated by year. Monthly revenue is then calculated twice: once using month names for a readable display and once using the numeric month so chronological sorting is reliable. The final sort orders each year-month combination correctly.

```
[77]: yearly_sales = (
      df_clean.groupby("Year")["Revenue"]
      .sum()
      )

yearly_sales
```

```
[77]: Year
2009    796648.500
2010   9216135.874
2011   9001425.464
Name: Revenue, dtype: float64
```

```
[78]: monthly_sales = (
      df_clean.groupby(["Year", "Month_Name"])["Revenue"]
      .sum()
      )

monthly_sales
```

```
[78]: Year  Month_Name
2009  December    796648.500
2010   April     641521.052
      August     654774.390
      December    746723.610
      February    531265.366
      January     622479.502
      July        617365.480
      June        677073.870
      March       763247.241
      May         613270.720
      November    1416697.202
      October     1080611.480
      September    851105.961
```

2011	April	492367.841
	August	703510.580
	December	432719.060
	February	497026.410
	January	558448.560
	July	680156.991
	June	689977.230
	March	682013.980
	May	722094.100
	November	1456145.800
	October	1069368.230
	September	1017596.682

Name: Revenue, dtype: float64

```
[79]: monthly_sales = (
        df_clean.groupby(["Year", "Month"])["Revenue"]
        .sum()
        .reset_index()
    )

monthly_sales
```

```
[79]:
```

	Year	Month	Revenue
0	2009	12	796648.500
1	2010	1	622479.502
2	2010	2	531265.366
3	2010	3	763247.241
4	2010	4	641521.052
5	2010	5	613270.720
6	2010	6	677073.870
7	2010	7	617365.480
8	2010	8	654774.390
9	2010	9	851105.961
10	2010	10	1080611.480
11	2010	11	1416697.202
12	2010	12	746723.610
13	2011	1	558448.560
14	2011	2	497026.410
15	2011	3	682013.980
16	2011	4	492367.841
17	2011	5	722094.100
18	2011	6	689977.230
19	2011	7	680156.991
20	2011	8	703510.580
21	2011	9	1017596.682
22	2011	10	1069368.230
23	2011	11	1456145.800

```
24 2011      12  432719.060
```

```
[80]: monthly_sales.sort_values(  
      ["Year", "Month"]  
      )
```

```
[80]:
```

	Year	Month	Revenue
0	2009	12	796648.500
1	2010	1	622479.502
2	2010	2	531265.366
3	2010	3	763247.241
4	2010	4	641521.052
5	2010	5	613270.720
6	2010	6	677073.870
7	2010	7	617365.480
8	2010	8	654774.390
9	2010	9	851105.961
10	2010	10	1080611.480
11	2010	11	1416697.202
12	2010	12	746723.610
13	2011	1	558448.560
14	2011	2	497026.410
15	2011	3	682013.980
16	2011	4	492367.841
17	2011	5	722094.100
18	2011	6	689977.230
19	2011	7	680156.991
20	2011	8	703510.580
21	2011	9	1017596.682
22	2011	10	1069368.230
23	2011	11	1456145.800
24	2011	12	432719.060

1.8 Compare sales by country

This section sums net revenue by country, ranks markets from largest to smallest, calculates the United Kingdom's share of total revenue, and lists the top countries after excluding the United Kingdom.

```
[81]: country_sales = (  
      df_clean.groupby("Country")["Revenue"]  
      .sum()  
      .sort_values(ascending=False)  
      )  
  
country_sales.head(10)
```

```
[81]: Country
      United Kingdom    1.614474e+07
      EIRE              6.099538e+05
      Netherlands      5.483307e+05
      Germany           4.119592e+05
      France            3.217334e+05
      Australia         1.664444e+05
      Switzerland      9.940306e+04
      Spain             9.101344e+04
      Sweden            8.777552e+04
      Denmark           6.445959e+04
      Name: Revenue, dtype: float64
```

```
[82]: uk_share = (
        country_sales["United Kingdom"] /
        country_sales.sum()
    ) * 100

    uk_share
```

```
[82]: np.float64(84.90879513033805)
```

```
[83]: country_sales.drop("United Kingdom").head(10)
```

```
[83]: Country
      EIRE              609953.780
      Netherlands      548330.700
      Germany           411959.161
      France            321733.390
      Australia         166444.420
      Switzerland      99403.060
      Spain             91013.440
      Sweden            87775.520
      Denmark           64459.590
      Belgium           63208.890
      Name: Revenue, dtype: float64
```

1.9 Identify top-performing products

Product revenue is initially ranked by description. The following check identifies postage-related descriptions, then creates `df_products` without any description containing “POSTAGE”. The final ranking shows product revenue without delivery charges; because it uses net revenue, returns and cancellations still reduce the relevant product totals.

```
[84]: product_sales = (
        df_clean.groupby("Description")["Revenue"]
        .sum()
        .sort_values(ascending=False)
```

```
)

product_sales.head(10)
```

```
[84]: Description
      REGENCY CAKESTAND 3 TIER          314045.02
      DOTCOM POSTAGE                  309844.10
      WHITE HANGING HEART T-LIGHT HOLDER 251603.12
      PARTY BUNTING                   147079.73
      JUMBO BAG RED RETROSPOT          145946.05
      ASSORTED COLOUR BIRD ORNAMENT    128550.42
      PAPER CHAIN KIT 50'S CHRISTMAS   116298.59
      POSTAGE                          110430.41
      CHILLI LIGHTS                    79905.13
      ROTATING SILVER ANGELS T-LIGHT HLDR 70969.95
      Name: Revenue, dtype: float64
```

```
[85]: df_clean[
        df_clean["Description"].str.contains(
            "POSTAGE",
            case=False,
            na=False
        )
    ]["Description"].value_counts()
```

```
[85]: Description
      POSTAGE          2079
      DOTCOM POSTAGE    1418
      Name: count, dtype: int64
```

```
[86]: df_products = df_clean[
        ~df_clean["Description"].str.contains(
            "POSTAGE",
            case=False,
            na=False
        )
    ].copy()
```

```
[87]: product_sales = (
        df_products.groupby("Description")["Revenue"]
        .sum()
        .sort_values(ascending=False)
    )

product_sales.head(10)
```

```
[87]: Description
      REGENCY CAKESTAND 3 TIER                314045.02
      WHITE HANGING HEART T-LIGHT HOLDER      251603.12
      PARTY BUNTING                          147079.73
      JUMBO BAG RED RETROSPOT                 145946.05
      ASSORTED COLOUR BIRD ORNAMENT           128550.42
      PAPER CHAIN KIT 50'S CHRISTMAS          116298.59
      CHILLI LIGHTS                          79905.13
      ROTATING SILVER ANGELS T-LIGHT HLDR     70969.95
      JUMBO BAG STRAWBERRY                    68489.89
      RABBIT NIGHT LIGHT                     66661.63
      Name: Revenue, dtype: float64
```

1.10 Prepare the RFM customer base

RFM analysis requires a known customer, so rows with missing customer IDs are removed. The dataset dimensions are displayed to show how many customer-linked transaction lines remain.

```
[88]: df_customers = df_clean[
      df_clean["Customer ID"].notna()
      ].copy()
```

```
[89]: df_customers.shape
```

```
[89]: (797815, 16)
```

1.10.1 Calculate Recency, Frequency, and Monetary value

The reference date is set to one day after the latest customer transaction. For each customer, the code calculates:

- **Recency:** days since the customer's most recent transaction.
- **Frequency:** number of distinct invoice identifiers.
- **Monetary:** total net revenue across the customer's transaction lines.

The aggregate column names are then replaced with these standard RFM labels, and the following views inspect typical rows and the highest-spending customers.

```
[90]: analysis_date = df_customers["InvoiceDate"].max() + pd.Timedelta(days=1)

      analysis_date
```

```
[90]: Timestamp('2011-12-10 12:50:00')
```

```
[91]: rfm = df_customers.groupby("Customer ID").agg({
      "InvoiceDate": lambda x: (
          analysis_date - x.max()
      ).days,
      "Invoice": "nunique",
```

```
    "Revenue": "sum"  
})
```

```
[92]: rfm.columns = [  
    "Recency",  
    "Frequency",  
    "Monetary"  
]
```

```
[93]: rfm.head()
```

```
[93]:
```

	Recency	Frequency	Monetary
Customer ID			
12346.0	326	17	-51.74
12347.0	2	8	4921.53
12348.0	75	5	2019.40
12349.0	19	5	4404.54
12350.0	310	1	334.40

```
[94]: rfm.sort_values(  
    "Monetary",  
    ascending=False  
) .head(10)
```

```
[94]:
```

	Recency	Frequency	Monetary
Customer ID			
18102.0	1	153	570380.61
14646.0	2	163	523342.07
14156.0	10	202	296063.44
14911.0	1	510	265757.91
17450.0	8	61	231390.55
13694.0	4	164	190020.84
17511.0	3	85	168491.62
12415.0	24	33	143269.29
16684.0	4	65	141502.25
15061.0	4	138	124961.98

```
[95]: rfm.head()
```

```
[95]:
```

	Recency	Frequency	Monetary
Customer ID			
12346.0	326	17	-51.74
12347.0	2	8	4921.53
12348.0	75	5	2019.40
12349.0	19	5	4404.54
12350.0	310	1	334.40

Interpretation note: This cell creates a sales-only customer subset by excluding

cancellations, but the RFM table above is not recalculated from it. As written, the RFM frequency and monetary values include cancellation transactions; monetary values are net of returns. The following ranking is therefore unchanged and serves as a check of that existing RFM table.

```
[96]: df_customers_sales = df_customers[
      ~df_customers["IsCancellation"]
    ].copy()
```

```
[97]: rfm.sort_values(
      "Monetary",
      ascending=False
    ).head(10)
```

```
[97]:
```

	Recency	Frequency	Monetary
Customer ID			
18102.0	1	153	570380.61
14646.0	2	163	523342.07
14156.0	10	202	296063.44
14911.0	1	510	265757.91
17450.0	8	61	231390.55
13694.0	4	164	190020.84
17511.0	3	85	168491.62
12415.0	24	33	143269.29
16684.0	4	65	141502.25
15061.0	4	138	124961.98

1.10.2 Score each RFM dimension

Customers are divided into five ranked groups (quintiles) for recency, frequency, and monetary value. Recency receives a higher score for fewer days since the last purchase, while frequency and monetary value receive higher scores for larger values. Ranking with `method="first"` breaks ties so the quintile calculation can assign every customer. The three scores are concatenated into a compact RFM score such as 555.

```
[99]: rfm["R_Score"] = pd.qcut(
      rfm["Recency"].rank(method="first"),
      5,
      labels=[5,4,3,2,1]
    )

rfm["F_Score"] = pd.qcut(
      rfm["Frequency"].rank(method="first"),
      5,
      labels=[1,2,3,4,5]
    )

rfm["M_Score"] = pd.qcut(
```

```
rfm["Monetary"].rank(method="first"),
5,
labels=[1,2,3,4,5]
)
```

```
[100]: rfm["RFM_Score"] = (
    rfm["R_Score"].astype(str)
    +
    rfm["F_Score"].astype(str)
    +
    rfm["M_Score"].astype(str)
)
```

```
[101]: rfm.head()
```

```
[101]:
```

	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
Customer ID							
12346.0	326	17	-51.74	2	5	1	251
12347.0	2	8	4921.53	5	4	5	545
12348.0	75	5	2019.40	3	3	4	334
12349.0	19	5	4404.54	4	3	5	435
12350.0	310	1	334.40	2	1	2	212

1.10.3 Assign business-friendly customer segments

The function maps score combinations to segments: high scorers on all three measures are **Champions**; customers with strong recency and frequency are **Loyal Customers**; highly recent customers are **Recent Customers**; less-recent but frequent buyers are **At Risk**; and all remaining customers are labelled **Lost Customers**. The next cells apply the rule to each customer and count the resulting segments.

```
[102]: def segment_customer(row):
    if row["R_Score"] >= 4 and row["F_Score"] >= 4 and row["M_Score"] >= 4:
        return "Champions"

    elif row["R_Score"] >= 3 and row["F_Score"] >= 3:
        return "Loyal Customers"

    elif row["R_Score"] >= 4:
        return "Recent Customers"

    elif row["R_Score"] <= 2 and row["F_Score"] >= 3:
        return "At Risk"

    else:
        return "Lost Customers"
```

```
[103]: rfm["Segment"] = rfm.apply(
        segment_customer,
        axis=1
    )
```

```
[104]: rfm["Segment"].value_counts()
```

```
[104]: Segment
Lost Customers      1919
Loyal Customers    1421
Champions           1301
At Risk              841
Recent Customers    457
Name: count, dtype: int64
```

1.11 Build the reporting tables and export the workbook

Three output tables are prepared: cleaned transaction lines (**sales**), customer RFM scores and segments (**customers**), and product-level quantity/revenue totals excluding postage (**products**). The final cell writes them to separate **Sales**, **Customers**, and **Products** sheets in **Retail_Analytics_Model.xlsx**.

```
[105]: sales = df_clean.copy()
```

```
[106]: customers = rfm.reset_index()
```

```
[107]: products = (
    df_products
    .groupby(["StockCode", "Description"])
    .agg(
        Total_Quantity=("Quantity", "sum"),
        Total_Revenue=("Revenue", "sum")
    )
    .reset_index()
)
```

```
[108]:
```

```
[112]: with pd.ExcelWriter("Retail_Analytics_Model.xlsx") as writer:
        sales.to_excel(
            writer,
            sheet_name="Sales",
            index=False
        )

        customers.to_excel(
            writer,
            sheet_name="Customers",
```

```
        index=False
    )

    products.to_excel(
        writer,
        sheet_name="Products",
        index=False
    )
```

[]: